

	DEPARTAMENT D'INFORMÀTICA		CURS 2025-2026
	2N DAW	M0616	Projecte
	Oriol Torra Tudela		Data: 22/05/2026

M0616-Projecte

ChessHub - Manual Tècnic

Versió 1.0

Data d'actualització: 22 de maig de 2026



Índex

1. Informació general.....	1
1.1. Historial de versions.....	1
1.2. Propòsit del document.....	1
2. Arquitectura del sistema.....	2
2.1. Visió general.....	2
2.2. Patrons de disseny utilitzats.....	4
2.3. Stack tecnològic.....	4
2.4. Components principals.....	5
3. Entorn tècnic.....	6
3.1. Requeriments de hardware.....	6
3.2. Requeriments de software.....	6
3.3. Navegadors compatibles.....	7
4. Configuració del sistema.....	8
4.1. Instal·lació de l'entorn de desenvolupament.....	8
4.2. Variables d'entorn.....	9
4.3. Desplegament a producció.....	10
5. Base de dades.....	11
5.1. Model de dades.....	11
5.2. Relacions principals.....	12
5.3. Scripts i còpies de seguretat.....	12
6. Estructura del codi.....	14
6.1. Organització del projecte.....	14
6.2. Convencions de nomenclatura.....	15
6.3. Components principals.....	15
7. APIs i interfícies.....	16
7.1. API interna (REST).....	16
7.2. Interfície de temps real (WebSocket).....	17
7.3. APIs i recursos externs.....	17
8. Seguretat.....	18
8.1. Mecanismes de seguretat.....	18
8.2. Polítiques i bones pràctiques.....	18
8.3. Registre i auditoria.....	19
9. Monitorització i manteniment.....	19
9.1. Logs.....	19
9.2. Monitorització i estat.....	19
9.3. Manteniment.....	19
10. Resolució de problemes.....	20
10.1. Problemes comuns.....	20
10.2. Suport.....	20
11. Annexos.....	21
11.1. Glossari de termes.....	21
11.2. Referències tècniques.....	21
11.3. Documentació complementària.....	21

1. Informació general

Camp	Valor
Nom del projecte	ChessHub — Plataforma d'escacs en línia
Tipus de document	Manual tècnic
Versió del document	1.0
Data d'última actualització	Maig de 2026
Autor i responsable	Oriol Torra Tudela
Centre	Institut Lacetània — CFGS Desenvolupament d'Aplicacions Web (2n)
Repositori de codi	github.com/OriolTorraTudela/chesshub
URL de producció	http://grup4.infla.cat

1.1. Historial de versions

Versió	Data	Descripció
0.5	Abril 2026	Primera versió funcional: autenticació, partides PvP i taulell.
0.8	Maig 2026	Puzzles, partides contra bot, anàlisi i desplegament a Kubernetes.
1.0	Maig 2026	Sistema d'amics, motor d'IA Stockfish, revisió de seguretat i documentació.

1.2. Propòsit del document

Aquest manual tècnic descriu el funcionament intern del sistema ChessHub. Va adreçat a personal tècnic amb coneixements de desenvolupament web que necessiti instal·lar, configurar, desplegar, mantenir o ampliar l'aplicació.

El document recull l'arquitectura del sistema, l'entorn tecnològic, els procediments d'instal·lació i desplegament, el model de dades, l'estructura del codi, les interfícies de programació (API), els mecanismes de seguretat i les pautes de manteniment. No és una guia d'ús per a l'usuari final, per a això cal consultar el Manual d'usuari.

2. Arquitectura del sistema

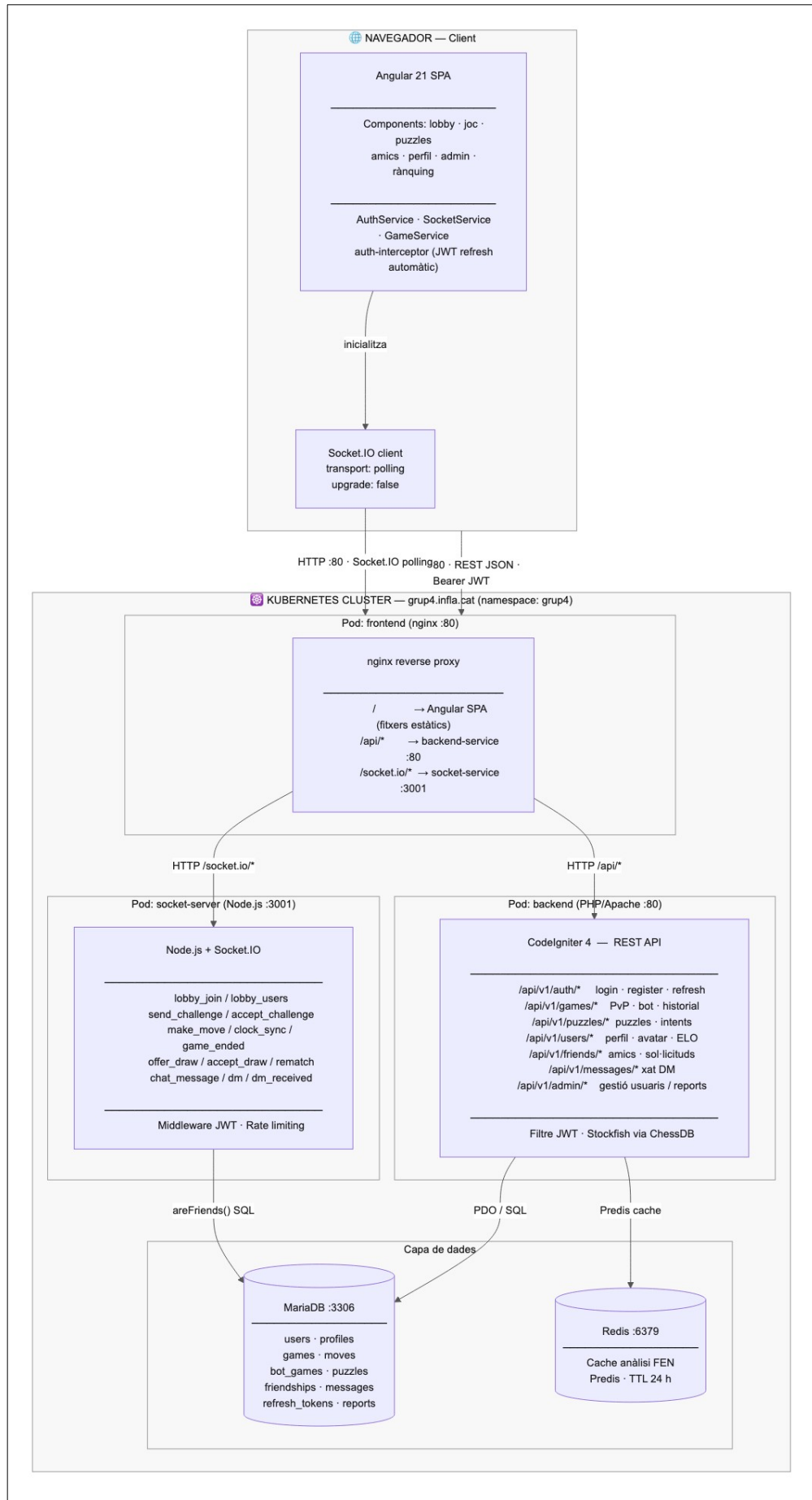
2.1. Visió general

ChessHub segueix una arquitectura client-servidor multicapa amb separació estricta entre la capa de presentació i la capa de negoci. El sistema està format per diversos serveis independents que s'executen en contenidors i es comuniquen mitjançant protocols estàndard (HTTP/REST i WebSocket).

Capes del sistema:

- **Capa de presentació (client):** aplicació d'una sola pàgina (SPA) feta amb Angular, executada al navegador de l'usuari.
- **Capa de negoci (servidor):** API REST feta amb Codelgniter 4 que conté tota la lògica de l'aplicació i l'accés a dades.
- **Servei de temps real:** servidor Node.js amb Socket.IO que sincronitza les partides en directe, el xat i els reptes entre jugadors.
- **Capa de dades:** base de dades relacional MariaDB, amb Redis com a memòria cau.
- **Motor d'intel·ligència artificial:** Stockfish, compilat a WebAssembly i executat dins el navegador del client.

Diagrama:



2.2. Patrons de disseny utilitzats

Patró	Aplicació al projecte
Model–Vista–Controlador (MVC)	El backend CodeIgniter organitza el codi en controladors i models; el frontend Angular separa components (vista) de serveis (lògica).
API REST	Comunicació client–servidor sense estat mitjançant recursos i verbs HTTP (GET, POST, PUT, DELETE).
Single Page Application	El frontend és una única aplicació que actualitza la vista sense recàrregues completes de pàgina.
Injecció de dependències	Tant Angular com CodeIgniter injecten serveis i models on toca.
Observador (Observable)	RxJS al frontend gestiona els fluxos asíncrons (peticions HTTP, esdeveniments del socket).
Autenticació sense estat (JWT)	El servidor no guarda sessions; cada petició s'autentica amb un token signat.

2.3. Stack tecnològic

Capa	Tecnologia	Versió
Frontend	Angular (TypeScript)	21
Frontend	Bootstrap	5.3
Frontend	chess.js (validació de moviments)	1.4
Frontend	socket.io-client	4.8
Frontend / IA	Stockfish (WebAssembly)	10
Backend	PHP	8.2
Backend	CodeIgniter 4	4.7
Backend	firebase/php-jwt	7.0
Backend	redis/redis (client Redis)	2.2
Temps real	Node.js	20
Temps real	Socket.IO + Express	4.8 / 5.2
Dades	MariaDB	11
Cau	Redis	7
Infraestructura	Docker, Kubernetes, Harbor, nginx	—

2.4. Components principals

Frontend

Aplicació Angular estructurada en components autònoms (standalone) i rutes amb càrrega mandrosa (lazy loading). Es compila a fitxers estàtics que serveix un servidor nginx.

Backend

API REST amb CodeIgniter 4 sobre PHP 8.2 i servidor Apache. Exposa els recursos sota el prefix /api/v1 i aplica filtres d'autenticació (JWT) i d'autorització (rol d'administrador).

Base de dades

MariaDB 11 emmagatzema usuaris, perfils, partides, moviments, puzzles, amistats, missatges i estadístiques. Redis s'utilitza com a memòria cau de resultats d'anàlisi.

Servei de temps real

Servidor Node.js amb Socket.IO que gestiona les sales de partida, la sincronització de moviments, el rellotge autoritzat del servidor, el xat, els reptes i la missatgeria entre amics.

Serveis externs

El sistema és autocontingut i no depèn de serveis externs crítics. El motor d'escacs Stockfish s'executa al navegador del client. Les imatges de les peces es carreguen d'un CDN públic d'escacs.

3. Entorn tècnic

3.1. Requeriments de hardware

L'aplicació s'executa sobre un clúster de Kubernetes. Cada servei declara els seus recursos mínims i màxims. La taula següent recull els requeriments del conjunt:

Recurs	Mínim recomanat
Processador	2 vÍrtues de CPU (vCPU)
Memòria RAM	4 GB (suma dels serveis i marge del sistema)
Emmagatzematge	10 GB (volum persistent de 2 GB per a MariaDB + imatges)
Xarxa	Connexió a Internet per descarregar les imatges del registre

La part d'IA (Stockfish) s'executa al dispositiu del client; per tant, no consumeix recursos del servidor. Es recomana que el client disposi d'un navegador modern amb suport de WebAssembly.

3.2. Requeriments de software

Àmbit	Software
Sistema operatiu (servidor)	Linux (clúster Kubernetes)
Orquestració	Kubernetes
Contenidors	Docker
Registre d'imatges	Harbor
Servidor web (frontend)	nginx
Servidor web (backend)	Apache + PHP 8.2
Base de dades	MariaDB 11
Memòria cau	Redis 7
Entorn de temps real	Node.js 20
Eines de desenvolupament	Node.js, npm, Angular CLI, Composer, Git

3.3. Navegadors compatibles

La interfície és compatible amb les versions recents de Google Chrome, Mozilla Firefox, Microsoft Edge i Safari. Cal que el navegador tingui JavaScript i WebAssembly habilitats (necessaris per al motor d'escacs).

4. Configuració del sistema

4.1. Instal·lació de l'entorn de desenvolupament

1. Clonar el repositori:

```
git clone https://github.com/OriolTorraTudela/chesshub.git  
cd chesshub
```

2. Backend (CodeIgniter): instal·lar les dependències amb Composer.

```
cd backend  
composer install  
cp env .env
```

3. Frontend (Angular): instal·lar les dependències amb npm.

```
cd frontend  
npm install
```

4. Servidor de temps real: instal·lar les dependències de Node.

```
cd socket-server  
npm install
```

5. Base de dades: crear la bd i importar l'esquema i dades inicials.

```
mysql -u root -p -e "CREATE DATABASE chesshub"  
mysql chesshub < database/chesshub_schema.sql  
mysql chesshub < database/migrations/002_friends.sql  
mysql chesshub < database/seeds/puzzles.sql
```

6. Posar en marxa els serveis:

backend: php spark serve

frontend: ng serve

temps real: node socket-server/server.js

4.2. Variables d'entorn

La configuració sensible es proporciona mitjançant variables d'entorn (mai no es desa al codi). Les principals són:

Variable	Servei	Descripció
DB_HOSTNAME / DB_DATABASE	Backend	Adreça i nom de la base de dades.
DB_USERNAME / DB_PASSWORD	Backend	Credencials d'accés a MariaDB.
JWT_SECRET	Backend i temps real	Clau secreta per signar i verificar els tokens JWT.
ALLOWED_ORIGINS	Backend	Orígens autoritzats per a les peticions.
REDIS_HOST	Backend	Adreça del servei Redis.
FRONTEND_URL	Temps real	Origen autoritzat per a les connexions de socket.

A producció, aquests valors es guarden en un Secret de Kubernetes (chesshub-secrets) i s'injecten als contenidors. El fitxer k8s/secrets.yaml mai no es puja al repositori; només s'hi puja k8s/secrets.example.yaml amb valors d'exemple.

4.3. Desplegament a producció

El desplegament es fa amb contenidors Docker publicats a un registre Harbor i orquestrats per Kubernetes (espai de noms grup4). El procediment és:

7. Actualitzar el codi al servidor: `git pull origin main`.

8. Construir les imatges Docker de cada servei modificat.

```
docker build -t kube0.lacetania.cat/grup4/chesshub-backend:latest ./backend
```

```
docker build -t kube0.lacetania.cat/grup4/chesshub-frontend:latest ./frontend
```

```
docker build -t kube0.lacetania.cat/grup4/chesshub-socket:latest ./socket-server
```

9. Publicar les imatges al registre Harbor: `docker push`.

10. Aplicar la nova versió al clúster.

```
kubectl rollout restart deployment/backend -n grup4
```

```
kubectl rollout restart deployment/frontend -n grup4
```

```
kubectl rollout restart deployment/socket-server -n grup4
```

Els manifestos de Kubernetes (carpeta k8s/) defineixen els Deployments, Services, el volum persistent de MariaDB i els Secrets de cada component.

5. Base de dades

5.1. Model de dades

ChessHub utilitza una base de dades relacional MariaDB amb 15 taules relacionades entre elles mitjançant claus forànies. El motor és InnoDB i el joc de caràcters utf8mb4.

Descripció de les taules:

Taula	Descripció
users	Dades d'autenticació de cada usuari (nom, email, contrasenya xifrada, rol).
profiles	Perfil públic: ELO, victòries, derrotes, taules, avatar, biografia.
refresh_tokens	Tokens de refresc actius per renovar la sessió.
themes	Temes visuals del taulell disponibles.
games	Partides entre jugadors (PvP): jugadors, estat, resultat, control de temps.
moves	Moviments de cada partida PvP, en notació SAN i UCI.
bot_games	Partides contra el bot: usuari, color, nivell, resultat.
bot_moves	Moviments de les partides contra el bot.
elo_history	Històric de variacions d'ELO de cada usuari.
game_analysis	Resultats de l'anàlisi post-partida (precisió, errors, etc.).
puzzles	Catàleg de problemes d'escacs (FEN, solució, dificultat).
puzzle_attempts	Intents de resolució de puzzles per usuari.
reports	Denúncies d'usuaris cap a altres jugadors.
friendships	Relacions d'amistat (sol·licituds pendents i amistats acceptades).
direct_messages	Missatges directes del xat entre amics.

5.2. Relacions principals

- Un usuari (users) té un perfil (profiles) relació 1:1.
- Una partida (games) referencia dos usuaris (jugador de blanques i de negres).
- Una partida té molts moviments (moves) relació 1:N.
- Un usuari té moltes entrades a l'històric d'ELO i molts intents de puzzle.
- Una amistat (friendships) relaciona dos usuaris (sol·licitant i destinatari).
- Les claus forànies utilitzen ON DELETE CASCADE: en esborrar un usuari, s'eliminen les seves dades dependents.

5.3. Scripts i còpies de seguretat

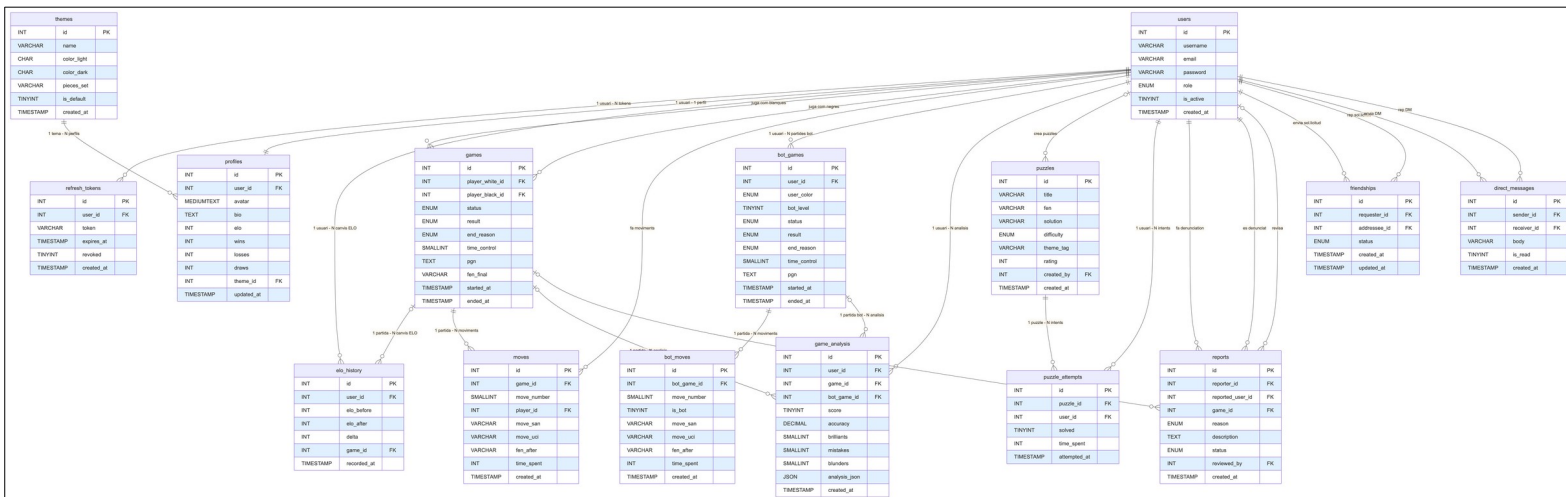
- database/chesshub_schema.sql script de creació de l'esquema inicial.
- database/migrations/002_friends.sql migració: taules d'amistats i missatges directes.
- database/seeds/puzzles.sql dades inicials: catàleg de puzzles verificats.

El sistema no utilitza triggers ni procediments emmagatzemats: tota la lògica resideix a la capa de negoci. Per fer una còpia de seguretat de la base de dades en producció s'utilitza mariadb-dump dins el contenidor de MariaDB.

```
kubect exec -n grup4 deployment/mariadb -- \
```

```
mariadb-dump -u chesshub -p chesshub > backup.sql
```

Diagrama:



El diagrama serà pujat al Github per poder veure'l millor, ja que per temes de dimencions aquí no es pot apreciar correctament.

6. Estructura del codi

6.1. Organització del projecte

El repositori s'organitza en una carpeta per servei, més les carpetes d'infraestructura i documentació:

chesshub/

 backend/ API REST (CodeIgniter 4)

 frontend/ Aplicació Angular

 socket-server/ Servidor de temps real (Node.js)

 database/ Esquema, migracions i dades inicials

 k8s/ Manifestos de Kubernetes

 docs/ Documentació del projecte

Estructura del backend

backend/app/

 Controllers/Api/ Controladors de l'API REST

 Models/ Models d'accés a dades

 Filters/ Filtres (JWT, administrador)

 Helpers/ Funcions auxiliars (jwt_helper)

 Config/ Configuració (Routes, Database...)

Estructura del frontend

frontend/src/app/

 core/services/ Serveis (auth, game, socket, engine)

 core/guards/ Guards de rutes

 core/interceptors/ Interceptor HTTP (token JWT)

 features/ Components per funcionalitat

 auth/ game/ puzzles/ profile/ friends/ ...

6.2. Convencions de nomenclatura

- Controladors del backend: nom en singular acabat en Controller (p. ex. GameController).
- Models del backend: nom de l'entitat acabat en Model (p. ex. UserModel).
- Components i serveis d'Angular: noms en PascalCase; fitxers en minúscules.
- Rutes de l'API: substantius en plural i verbs HTTP (GET /games, POST /games).

6.3. Components principals

Serveis del frontend

Servei	Responsabilitat
AuthService	Registre, inici i tancament de sessió, gestió dels tokens.
GameService	Crides a l'API: partides, puzzles, perfils, amics, missatges.
SocketService	Connexió amb el servidor de temps real (Socket.IO).
EngineService	Control del motor d'IA Stockfish (Web Worker, protocol UCI).

Controladors del backend

Controlador	Responsabilitat
AuthController	Registre, login, refresc i tancament de sessió.
UserController	Perfils, estadístiques, rànkings, denúncies.
GameController	Partides PvP: creació, unió, moviments, finalització, històric.
BotGameController	Partides contra el bot.
PuzzleController	Catàleg de puzzles i intents de resolució.
AnalysisController	Anàlisi post-partida amb memòria cau Redis.
FriendController	Sistema d'amics: cerca, sol·licituds, llista.
MessageController	Missatgeria directa entre amics.
AdminController	Panell d'administració (usuaris, puzzles, denúncies).

7. APIs i interfícies

7.1. API interna (REST)

El backend exposa una API REST sota el prefix /api/v1. Totes les respostes són en format JSON amb una estructura uniforme:

```
{ "status": "success", "data": { ... } }
```

```
{ "status": "error", "message": "descripció de l'error" }
```

Les peticions amb cos envien dades en JSON. Els endpoints protegits requereixen una capçalera Authorization: Bearer <token>.

Principals grups d'endpoints:

Recurs	Mètode i ruta	Descripció
Autenticació	POST /auth/register	Registre d'un usuari nou.
Autenticació	POST /auth/login	Inici de sessió.
Autenticació	POST /auth/refresh	Renovació del token d'accés.
Usuaris	GET /users/me	Dades de l'usuari autenticat.
Usuaris	GET /leaderboard	Rànquing global per ELO.
Partides	POST /games	Crear una partida PvP.
Partides	POST /games/{id}/move	Registrar un moviment.
Partides	GET /games/history	Historial de partides de l'usuari.
Bot	POST /bot-games	Crear una partida contra el bot.
Puzzles	GET /puzzles	Llistar puzzles per dificultat.
Anàlisi	POST /analysis/game/{id}	Analitzar una partida acabada.
Amics	GET /friends/search	Cercar usuaris per afegir-los.
Missatges	POST /messages/{id}	Enviar un missatge directe.
Admin	GET /admin/stats	Estadístiques (només administradors).

7.2. Interfície de temps real (WebSocket)

El servidor Socket.IO gestiona els esdeveniments en directe. Els principals són:

- **join_game / make_move / move_made** unió a una partida i sincronització de moviments.
- **clock_sync** difusió del rellotge autoritzat del servidor.
- **game_start / game_ended** inici i final de partida.
- **chat_message / dm** xat de partida i missatgeria entre amics.
- **send_challenge / accept_challenge** sistema de reptes del lobby i d'amics.
- **rematch_offer / rematch_accepted** sol·licitud de revenja.

7.3. APIs i recursos externs

El sistema és autocontingut. El motor d'IA Stockfish s'executa al client (no és un servei extern). L'únic recurs extern és un CDN públic d'on es carreguen les imatges SVG de les peces d'escacs.

8. Seguretat

8.1. Mecanismes de seguretat

- **Autenticació amb JWT:** en iniciar sessió es genera un token d'accés (vigència de 8 hores) i un token de refresc (7 dies), signats amb l'algoritme HS256.
- **Xifratge de contrasenyes:** les contrasenyes es desen amb un hash bcrypt; mai no es guarden en text pla.
- **Autorització per rols:** un filtre comprova el token a les rutes protegides i un segon filtre restringeix el panell d'administració als usuaris amb rol d'administrador.
- **Protecció contra injecció SQL:** tot l'accés a dades es fa amb el constructor de consultes de CodeIgniter, que utilitza consultes preparades i escapa els paràmetres.
- **Protecció contra XSS:** Angular escapa per defecte el contingut interpolat a les plantilles.
- **Limitació d'intents:** els endpoints de login i registre apliquen un control d'intents per adreça IP per mitigar atacs de força bruta.

8.2. Polítiques i bones pràctiques

- **Gestió de secrets:** les credencials i la clau JWT s'injecten per variables d'entorn des d'un Secret de Kubernetes; no figuren al codi ni al repositori.
- **Accés al servidor:** l'administració del servidor es fa per SSH en un port no estàndard; no s'utilitza FTP.
- **Validació d'entrada:** el backend valida les dades de registre i de les operacions sensibles abans de processar-les.
- **Verificació d'autoria:** cada operació sobre una partida comprova que qui la fa n'és participant; el rendiment i el final de partida es validen al servidor.

8.3. Registre i auditoria

Les denúncies entre usuaris es desen a la taula reports per a la seva revisió des del panell d'administració. Els errors de l'aplicació queden registrats als logs del framework.

9. Monitorització i manteniment

9.1. Logs

Cada servei genera els seus propis registres:

- **Backend:** CodeIgniter escriu els logs a backend/writable/logs/, amb un fitxer per dia.
- **Contenidors:** la sortida estàndard de cada contenidor és accessible amb kubectl logs.

```
kubectl logs -n grup4 deployment/backend
```

```
kubectl logs -n grup4 deployment/socket-server
```

9.2. Monitorització i estat

L'estat dels serveis es consulta amb les eines de Kubernetes:

```
kubectl get pods -n grup4
```

```
kubectl rollout status deployment/backend -n grup4
```

El servidor de temps real exposa un endpoint /health que retorna l'estat del servei i el nombre de sales actives.

9.3. Manteniment

- Actualització de codi: git pull, reconstrucció de les imatges i rollout restart.
- Neteja periòdica d'imatges Docker antigues al servidor i al registre Harbor.
- Còpies de seguretat de la base de dades amb mariadb-dump.

10. Resolució de problemes

10.1. Problemes comuns

Problema	Causa probable	Solució
Un pod no arrenca	Imatge no disponible o error de configuració.	Revisar kubectl describe pod i kubectl logs; comprovar el Secret.
Error 401 a l'API	Token JWT caducat o absència de token.	Renovar la sessió; verificar que JWT_SECRET coincideix al backend i al socket.
Les partides no se sincronitzen	Servei de temps real caïgut.	Comprovar el pod socket-server i l'endpoint /health.
El bot no respon	El navegador no ha carregat el motor.	Comprovar que els fitxers de /engine/ es serveixen correctament.
Error de connexió a la BD	MariaDB no disponible o credencials incorrectes.	Revisar el pod mariadb i les variables DB_*.

10.2. Suport

Responsable tècnic: Oriol Torra Tudela — oriol.torra24@lacetania.cat

Codi font i incidències: github.com/OriolTorraTudela/chesshub

11. Annexos

11.1. Glossari de termes

Terme	Definició
SPA	Single Page Application. Aplicació web d'una sola pàgina.
API REST	Interfície de programació basada en recursos i verbs HTTP.
JWT	JSON Web Token. Token signat per autenticar peticions sense estat.
WebSocket	Protocol de comunicació bidireccional en temps real.
WebAssembly	Format binari que permet executar codi d'alt rendiment al navegador.
FEN	Notació estàndard per descriure una posició d'escacs.
UCI	Protocol de comunicació amb motors d'escacs.
ELO	Sistema de puntuació de la força dels jugadors.
Container	Unitat de software aïllada i portable (Docker).
Kubernetes	Sistema d'orquestració de contenidors.

11.2. Referències tècniques

- Documentació d'Angular: angular.dev
- Documentació de CodeIgniter 4: codeigniter.com/user_guide
- Documentació de Socket.IO: socket.io/docs
- Documentació de Kubernetes: kubernetes.io/docs
- Stockfish (motor d'escacs): stockfishchess.org

11.3. Documentació complementària

- Manual d'usuari de ChessHub.
- Pla de tests (docs/PLA_DE_TESTS.md).
- Memòria del projecte.